Lab: Building a REST API with Cloud Functions (Node.js)

Overview

In this lab, you will build and deploy a simple REST API using **Google Cloud Functions (HTTP-triggered)** and **Node.js**.
The API will expose three endpoints:

- `GET /items` – return all items
  - Returns 200 status code
- `GET /items/:id` – return a single item by ID
  - Returns 200 status code on success
  - Returns 404 status code when an item is not found
- `POST /items` – create a new item
  - Returns 201 status code on success
  - Returns 400 status code on success if missing `name` property

This lab reinforces:

- HTTP methods and REST conventions
- Path parameters and request bodies
- Basic API design
- Deploying and testing Cloud Functions

> **Assumption:** You are working in **Cloud Shell**, using **Node.js**, and deploying **HTTP Cloud Functions**.

Learning Objectives

By the end of this lab, you will be able to:

- Create an HTTP-based REST API using Cloud Functions

- Handle multiple routes in a single function
- Parse path parameters and JSON request bodies
- Test API endpoints using a browser and `curl`

Scenario

You are building a simple backend service for an application that manages a list of items. For now, the data will be stored **in memory** (no database yet). Each item has:

```
{
  "id": number,
  "name": string
}
```

Step 1: Project Setup

1. Open **Cloud Shell**.
2. Create a new project folder:

```
mkdir rest-api-lab
cd rest-api-lab
```

1. Open the new folder in VSCode
2. Connect to your Google Cloud project:

```
npm init -y
```

1. Initialize a Node.js project:

```
npm init -y
```

1. Install dependencies:

```
npm install express && npm install --save-dev @google-cloud/functions-framework
```

1. Modify `scripts` property in `package.json`

package.json

```
"scripts": {
    "test": "...",
    "start": "npx functions-framework --target=studentsApi
}
```

Step 2: Create the API (`index.js`)

Create a file named `index.js`.

Implementation

Try to write it yourself, then look at the code solution to check your code.

Code Solutions

index.js

```
const express = require('express');
const app = express();

app.use(express.json());

// In-memory data store
let items = [
  { id: 1, name: 'Keyboard' },
  { id: 2, name: 'Mouse' }
];

// GET /items — return all items
app.get('/items', (req, res) => {
  res.status(200).json(items);
});

// GET /items/:id — return item by ID
app.get('/items/:id', (req, res) => {
  const id = parseInt(req.params.id);
  const item = items.find(i => i.id === id);

  if (!item) {
    return res.status(404).json({ message: 'Item not found' });
  }

  res.status(200).json(item);
});

// POST /items — create a new item
app.post('/items', (req, res) => {
  const { name } = req.body;

  if (!name) {
    return res.status(400).json({ message: 'Name is required' });
  }

  const newItem = {
    id: items.length + 1,
    name
  };

  items.push(newItem);
  res.status(201).json(newItem);
});

// Export for Cloud Functions
exports.api = app;
```

Step 3: Test the API

You will test all endpoints using **curl**.

Postman Setup

1. In the terminal, execute the following command to start your test server:

```
npm run start
```

1. Open a new terminal

You will now use the `curl` command to test the different scenarios.

1. GET All Items

- Method: **GET**
- URL:

```
{{ baseUrl }}/items
```

- Terminal Command:

```
curl {{ baseUrl }}/items
```

Expected response:

```
[
  { "id": 1, "name": "Keyboard" },
  { "id": 2, "name": "Mouse" }
]
```

2. GET Item by ID

- Method: **GET**
- URL:

```
{{ baseUrl }}/items/1
```

- Terminal Command:

```
curl {{ baseUrl }}/items/1
```

Expected response:

```
{ "id": 1, "name": "Keyboard" }
```

3. POST a New Item

- Method: **POST**
- URL:

```
{{ baseUrl }}/items
```

- Headers:

```
Content-Type: application/json
```

- Body → **raw** → **JSON**:

```
{
  "name": "Monitor"
}
```

- Terminal Command to make a POST request with `curl`:

```
curl -X POST FUNCTION_URL/items \
  -H "Content-Type: application/json" \
  -d '{ "name": "Monitor" }'
```

Expected response:

```
{ "id": 3, "name": "Monitor" }
```

Step 4: Verify Error Handling

Try the following:

- Request an ID that does not exist
- Send a POST request with no `name`

Observe the HTTP status codes and error messages.

Step 5: Deploy the Cloud Function (Gen 2)

This lab uses **Cloud Functions (2nd gen)**.

Deploy the function with the following command:

```
gcloud functions deploy api \
  --gen2 \
  --runtime=nodejs20 \
  --region=us-central1 \
  --source=. \
  --entry-point=api \
  --trigger-http \
  --allow-unauthenticated
```

Deliverables

Submit:

1. The **Cloud Function Gen 2 URL**
2. A screenshot showing the API requests and responses when using curl for the following API calls:
   - `GET /items`
   - `GET /items/:id`
   - `POST /items`
3. Purdue GitHub URL

All of these can be put into a word document and uploaded to the lab.

Challenges (Do not need for the lab)

⭐ Challenge 1: DELETE Endpoint

Add:

- `DELETE /items/:id` – removes an item by ID

Return:

- `204 No Content` on success
- `404 Not Found` if the item does not exist

⭐ Challenge 2: PUT Endpoint

Add:

- `PUT /items/:id` – updates an existing item's name

Validate input and return appropriate status codes.

⭐ Challenge 3: Query Parameters

Modify `GET /items` to support:

```
/items?name=keyboard
```

Return only matching items.

⭐ Challenge 4: Input Validation

Ensure:

- `name` must be at least 3 characters long
- Reject invalid requests with `400 Bad Request`

⭐ Challenge 5: Refactor for Scale

Move:

- Routes into a separate file
- Data logic into its own module

Explain (in comments) why this matters for larger APIs.

Reflection Questions

1. Why is using proper HTTP status codes important?
2. What limitations exist when using in-memory data storage?